

SD Programming 101 - Routines and an Application

Lessons 7 to 10: subroutines, functions, error handling, and a small application put together.

This page continues SD Programming 101.

7. A subroutine

A subroutine is a separate program, called by name, that takes arguments and returns values through them. You write it as its own source record and compile it:

```
subroutine get.field(result, file.var, id, field.no)
  read rec from file.var, id then
    result = rec<field.no>
  end else
    result = ''
  end
end
```

```
basic bp get.field
```

Notice: a subroutine starts with `subroutine` where a program starts with `program`. It does not `run` — you call it from another program.

```
program test.get.field
  open 'customers' to f.cust else stop 'no file'
  call get.field(name, f.cust, '1001', 1)
  print 'name is ' : name

  call get.field(phone, f.cust, '1001', 2)
  print 'phone is ' : phone
end
```

```
basic bp test.get.field
run bp test.get.field
```

```
name is Acme Supplies
phone is 555-1234
```

`call` runs the subroutine. Arguments are passed **by reference**: the subroutine sets `result` and the caller sees it in `name`. This is how subroutines return values — there is no `return value` statement for subroutines.

A subroutine must be compiled before the program that calls it. If `get.field` is not compiled, the calling program gets *Subroutine not found* at run time, not at compile time.

8. A function

A function is like a subroutine, but it returns a value in an expression rather than through an argument:

```
function fmt.phone(number)
  if len(number) = 7 then
    fmt.phone = number[1,3] : '-' : number[4,3]
  end else
    fmt.phone = number
  end
end
```

```
basic bp fmt.phone
```

A function assigns its return value to **its own name**. That is the `fmt.phone = ...` line — it is not a variable called `fmt.phone`, it is the function saying what to return.

A caller has to declare it with `deffun` before using it. Without that line, the compiler has no way to know `fmt.phone(raw)` is a call rather than a matrix reference, and says so in those terms: *Matrix FMT.PHONE is not referenced in a DIM statement*. Subroutines called with `call` need no declaration; functions do.

```
program test.fmt.phone
  deffun fmt.phone(number) calling 'FMT.PHONE'
  open 'customers' to f.cust else stop 'no file'
  read rec from f.cust, '1001' then
    raw = rec<2>
    print 'raw      : ' : raw
    print 'formatted: ' : fmt.phone(raw)
  end
end
```

```
basic bp test.fmt.phone
run bp test.fmt.phone
```

```
raw      : 5551234
formatted: 555-1234
```

`number[1,3]` is a **substring**: start at position 1, take 3 characters. `number[4,3]` starts at position 4 and takes 3 more. The square brackets are the shorthand for the `substr()` function.

9. Error handling

File operations can fail. The `on error` clause catches failures that are not about the record being missing — disk full, file locked by another session, network error:

```
program safe.write
  open 'customers' to f.cust else stop 'no file'

  print 'id?' :
  input id
  print 'name?' :
  input name

  rec = name : @fm : '' : @fm : ''

  write rec on f.cust, id on error
    print 'write failed, status ' : status()
    stop
  end
  print 'saved'
end
```

```
basic bp safe.write
run bp safe.write
```

```
id? 1003
name? Test Co
saved
```

The `on error` clause fires when the write fails for a system reason. The `then/else` on `write` is about the record — but `write` has no `then/else`, because writing always succeeds unless something is broken. The `on error` is where that broken-something arrives.

`status()` **carries the code**, and it is what to read inside an `on error` block. Read it on the line you care about rather than three lines later: the next operation that sets a status overwrites it.

There is no system variable holding the error *text*. To turn a code into a sentence, call the catalogued subroutine

`!ERRTEXT`:

```
call "!ERRTEXT", message, status()
print 'write failed: ' : message
```

`@system.return.code` is a different thing and is worth not confusing with this one: it carries the result of the last `execute` command, not of the last file operation.

10. Putting it together: a small application

This program ties everything together. It is a menu-driven customer manager: list, add, view and delete. It uses subroutines, select lists, file I/O, error handling, and a loop.

```
program customer.manager
  open 'customers' to f.cust else
    print 'cannot open customers - create it first'
    stop
  end

  loop
    print
    print 'l)ist  a)dd  v)iew  d)elele  q)uit'
    print 'choice?' :
    input choice

    begin case
      case choice = 'l'
        gosub list.all

      case choice = 'a'
        gosub add.one

      case choice = 'v'
        gosub view.one
```

```

        case choice = 'd'
            gosub delete.one

        case choice = 'q'
            exit

        case 1
            print 'not a choice'
        end case
    repeat

    print 'bye'
    stop

* ---- list ----
list.all:
    select f.cust to 2
    cnt = 0

    print 'id' : ' ' : 'name' : ' ' : 'city'
    print '---' : ' ' : '----' : ' ' : '----'

    loop
        readnext id from 2 else exit
        read rec from f.cust, id then
            print id : ' ' : rec<1> : ' ' : rec<3>
            cnt += 1
        end
    repeat

    clearselect 2
    print cnt : ' record(s)'
    return

* ---- add ----
add.one:
    print 'new id?' :
    input id
    print 'name?' :
    input name
    print 'phone?' :
    input phone
    print 'city?' :
    input city

    rec = name : @fm : phone : @fm : city

```

```

write rec on f.cust, id on error
  print 'write failed, status ' : status()
  return
end
print 'saved'
return

* ---- view ----
view.one:
  print 'id?' :
  input id
  read rec from f.cust, id then
    print 'name : ' : rec<1>
    print 'phone : ' : rec<2>
    print 'city : ' : rec<3>
  end else
    print 'not found'
  end
  return

* ---- delete ----
delete.one:
  print 'id?' :
  input id
  print 'are you sure? (y/n)' :
  input confirm
  if confirm # 'y' then
    print 'cancelled'
    return
  end

  delete f.cust, id on error
    print 'delete failed, status ' : status()
    return
  end
  print 'deleted'
  return
end

```

```

basic bp customer.manager
run bp customer.manager

```

```

1)list a)dd v)view d)delete q)uit
choice? 1

```

```

id name city
--- ----
1001 Acme Supplies Springfield
1002 Widget Corp Shelbyville
2 record(s)

l)ist a)dd v)iew d)elele q)uit
choice? v
id? 1001
name  : Acme Supplies
phone : 555-1234
city  : Springfield

l)ist a)dd v)iew d)elele q)uit
choice? q
bye

```

WHAT IS IN THIS PROGRAM

FEATURE	WHERE
<code>loop ... repeat</code>	the main menu loop
<code>begin case</code> / <code>case</code> / <code>end case</code>	dispatching on the menu choice
<code>gosub</code> / <code>return</code>	internal subroutines — labels inside the program
<code>select</code> / <code>readnext</code> / <code>clearselect</code>	walking the file for the list
<code>read</code> / <code>then</code> / <code>else</code>	reading a record
<code>write</code> / <code>on error</code>	writing a record safely
<code>delete</code> / <code>on error</code>	deleting a record
<code>@fm</code>	building a dynamic array
<code>rec<1></code>	extracting fields
<code>exit</code>	leaving the main loop
<code>#</code>	not-equal comparison
<code>+=</code>	increment

The `gosub` subroutines are **internal** — they are labels in the same program, not separate programs. They share every variable in the program. For subroutines with their own scope, see *SD Basic - Modern Program Structure*.

Cataloguing

Running a program with `run bp name` works, but typing `run bp` every time is tedious. **Cataloguing** gives a program a name you can type directly:

```
basic bp customer.manager
catalog bp customer.manager
```

```
customer.manager
```

```
l)list a)dd v)iew d)elele q)uit
choice?
```

A catalogued program can be called from any account that can reach it. The catalogue puts an entry in the VOC — the same file that maps `list`, `select` and the built-in verbs. See *SD TCL - Programs and the Catalogue*.

Where to go next

Each section above linked to a reference page. Here they are together, in the order the tutorial uses them:

TOPIC	REFERENCE PAGE
Program structure	SD Basic - Program Structure
Variables, loops, conditions	SD Basic - Program Control
Math functions	SD Basic - Math Functions
String functions	SD Basic - String Functions
Dynamic arrays	SD Basic - Dynamic Arrays
Data conversion	SD Basic - Data Conversion
File handling	SD Basic - File Handling

TOPIC	REFERENCE PAGE
Select lists	SD Basic - Select Lists
Locks and transactions	SD Basic - Locks and Transactions
Modern program structure	SD Basic - Modern Program Structure
Compiling and cataloguing	SD TCL - Programs and the Catalogue
File system concepts	SD File System Concepts
Standard subroutines	SD Standard Subroutines

The syntax cards at the end of the set list every statement and function alphabetically:

SD Basic - Syntax	every SDBasic name, with its syntax
SD TCL - Syntax	every TCL verb, with its syntax